

AI Usage Transcript

COMS3011A Lab 1 - Todo Application

Tool Used: ChatGPT (OpenAI)

Constraints Provided

Use Next.js, TypeScript, Prisma, and SQLite. Follow the COMS3011A Lab 1 requirements. Implement overdue tasks as a derived property rather than storing 'Overdue' as a status. Use an archive flag instead of deleting tasks. Provide code incrementally so it can be reviewed and understood.

Example 1: Database Design

The AI suggested a Prisma Task model containing title, description, topic, dueDate, status, archived, createdAt, and updatedAt fields. The design was accepted because archived tasks remain stored, overdue status is derived from existing data, and the schema satisfies the project requirements.

Example 2: Correcting an AI Suggestion

Initially, archived tasks disappeared from the active list without any way to view them. After reviewing the lab specification, it was identified that archived tasks must remain viewable. The solution was adjusted by creating an Archived Tasks page that displays tasks where archived = true.

Example 3: Debugging

When newly created tasks did not appear, investigation using Prisma Studio revealed that records were not being inserted. The issue was traced to the absence of a POST route. A POST handler was implemented, allowing tasks to be created and persisted.

Example 4: Overdue Task Logic

The AI suggested deriving overdue status at runtime by comparing the due date with the current date and ensuring the task is not marked Complete. This approach was accepted because it matches the specification.

Example 5: Testing

The AI assisted with Jest configuration and example tests for TaskForm and TaskList. The tests were reviewed, corrected where necessary, and executed locally.

Reflection

AI was used for architecture guidance, debugging, Prisma and Next.js implementation support, test generation, and documentation drafting. All code was reviewed and integrated manually. Several AI suggestions were refined to satisfy the project requirements.